time you face this problem in your application. This is very error prone.

An already better solution is to provide a wrapper class that internally uses those two structures: it gives you encapsulation, isolation and as a result you are able to write only once the code that syncs both structures on, say, add or merge (and so you gain reusability) and also, even more important, you are able to unit-test your class in isolation and hence have a reasonable guarantee that it’s going to do its job without bugs every time you use it in your application (that is, of course, provided you do write good and thorough unit-tests, acing the edge cases and challenging the behavior of your class in all possible contexts).

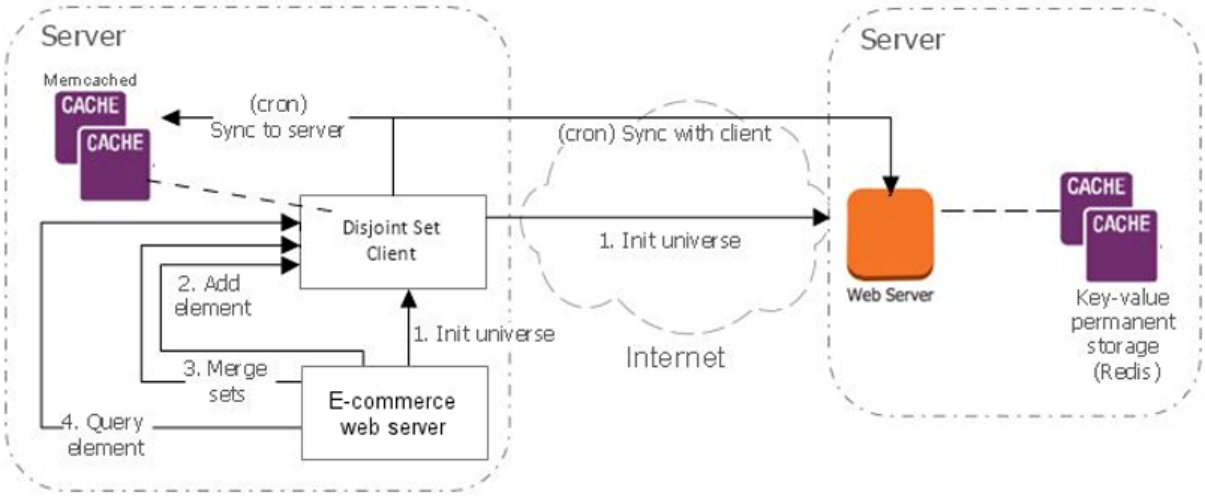


Figure 5.3 A possible design for an application using disjoint set. The disjoint set client can be anything ranging from a library to a REST client. The purpose of the (thin) client here is to be an interface between the in-memory storage (the Memcached node in the picture) and the server, which has persistent storage. The server can be a web server, but it could also just